

- [EKS Deep-Dive Part 3: Networking, Node Lifecycle, Security & Fluent Bit](#)
 - [1. NETWORK PLUGIN DECISION: VPC CNI vs CALICO](#)
 - [Decision Matrix](#)
 - [Recommendation for Private Cluster + Aurora](#)
 - [2. IP POOL & ASSIGNMENT — UNDER THE HOOD](#)
 - [How VPC CNI Assigns IPs to Pods](#)
 - [3. CONTROL PLANE & WORKER NODE LIFECYCLE](#)
 - [3.1 Control Plane Creation \(AWS-Managed\)](#)
 - [3.2 Worker Node Creation & Joining](#)
 - [4. SECURITY DEEP-DIVE](#)
 - [4.1 Network Security Layers](#)
 - [4.2 Pod Security \(Pod Security Standards\)](#)
 - [4.3 IRSA Security for Aurora Access](#)
 - [5. FLUENT BIT LOG FORWARDING](#)
 - [Architecture](#)
 - [Fluent Bit DaemonSet Configuration](#)
 - [Fluent Bit DaemonSet Deployment](#)
 - [Fluent Bit IRSA \(Private Cluster — No Internet\)](#)
 - [Fluent Bit During EKS Upgrade](#)

EKS Deep-Dive Part 3: Networking, Node Lifecycle, Security & Fluent Bit

1. NETWORK PLUGIN DECISION: VPC CNI vs CALICO

Decision Matrix

Aspect	AWS VPC CNI (Default)	Calico
Pod IP source	Real VPC IPs from subnet	Overlay network (VXLAN/IPIP) — virtual IPs

Aspect	AWS VPC CNI (Default)	Calico
Performance	Native VPC speed, no encapsulation overhead	~5-10% overhead due to encapsulation
IP consumption	Consumes VPC subnet IPs (can exhaust)	Uses private CIDR, no VPC IP consumption
Network Policies	Limited (needs Calico addon for policies)	Full-featured built-in
Security Groups	SG per pod supported (SGP feature)	Not applicable — uses Calico policies
AWS service access	Direct — pods have VPC IPs, talk to RDS/S3 natively	Requires NAT/masquerade for AWS services
Troubleshooting	Easy — pods visible in VPC flow logs	Harder — overlay traffic not in VPC flow logs
Best for	AWS-native workloads, RDS/Aurora integration	Large clusters needing >IP capacity, multi-cloud

Recommendation for Private Cluster + Aurora

Use VPC CNI + Calico for network policies. Reason:

- Pods get real VPC IPs → direct connectivity to Aurora on port 5432
- No NAT/masquerade needed for AWS services
- Add Calico **only for NetworkPolicy enforcement** (not as CNI)

```
# Install Calico for NetworkPolicy only (VPC CNI remains the CNI)
kubectl apply -f https://raw.githubusercontent.com/aws/amazon-vpc-cni-k8s/master/config/master/calico-operator.yaml
kubectl apply -f https://raw.githubusercontent.com/aws/amazon-vpc-cni-k8s/master/config/master/calico-crs.yaml
```

2. IP POOL & ASSIGNMENT — UNDER THE HOOD

How VPC CNI Assigns IPs to Pods

```
EC2 Worker Node (e.g., m5.xlarge)

Primary ENI (eth0): 10.0.10.5 (node IP)
├─ Secondary IP: 10.0.10.6 → assigned to Pod A
├─ Secondary IP: 10.0.10.7 → assigned to Pod B
└─ Secondary IP: 10.0.10.8 → warm pool (ready)

Secondary ENI (eth1): 10.0.10.20
├─ Secondary IP: 10.0.10.21 → assigned to Pod C
├─ Secondary IP: 10.0.10.22 → assigned to Pod D
└─ Secondary IP: 10.0.10.23 → warm pool

ipamd (L-IPAM daemon) manages the IP warm pool
WARM_IP_TARGET=2 (keep 2 IPs ready for new pods)
```

Step-by-step IP assignment:

- 1. **Node boots** → VPC CNI's **ipamd** daemon starts
- 2. **ipamd** attaches ENIs to the node and requests secondary IPs from the VPC subnet
- 3. Each EC2 instance type has ENI/IP limits:

Instance	Max ENIs	IPs per ENI	Max Pods (IPs)
t3.medium	3	6	17
m5.large	3	10	29
m5.xlarge	4	15	58
m5.2xlarge	4	15	58

- 1. When a pod is scheduled → kubelet asks CNI for an IP → **ipamd** assigns from warm pool
- 2. When pod terminates → IP returns to warm pool → eventually released back to subnet

Prefix delegation (solve IP exhaustion):

```
# Enable prefix delegation – each ENI slot gets a /28 prefix (16 IPs)
instead of 1 IP
kubectl set env daemonset/aws-node -n kube-system \
  ENABLE_PREFIX_DELEGATION=true \
```

```
WARM_PREFIX_TARGET=1
```

```
# Result: m5.xlarge goes from 58 pods to 110 pods per node  
# Uses fewer ENI slots but assigns /28 CIDR blocks
```

Subnet sizing for private clusters:

```
Cluster: 10 worker nodes × 58 pods = 580 pod IPs needed  
Nodes themselves: 10 IPs  
Buffer (warm pool): ~50 IPs  
During upgrade (2x nodes): 1200+ IPs needed
```

```
Recommendation: /19 subnets (8190 IPs) per AZ minimum  
                /18 subnets (16382 IPs) for large clusters
```

3. CONTROL PLANE & WORKER NODE LIFECYCLE

3.1 Control Plane Creation (AWS-Managed)

```
aws eks create-cluster
```



```
AWS creates (fully managed, you don't see):
```

- 3x etcd nodes (across 3 AZs)
- 2x API server instances (behind NLB)
- Controller Manager
- Scheduler
- Cloud Controller Manager
- EKS-managed ENIs in YOUR subnets
 (for API server ↔ worker communication)

```
API Server Endpoint:
```

- Private: ENI in your VPC (10.0.x.x)
- Public: disabled (private cluster)

```
OIDC Provider: created for IRSA
```

```
Cluster Security Group: auto-created
```

```
Cluster Role: manages AWS resources
```

Security at control plane level:

- **Envelope encryption:** Enable KMS for etcd secrets encryption

```
aws eks create-cluster --name my-cluster \  
  --encryption-config '["provider":  
{ "keyArn": "arn:aws:kms:...", "resources": ["secrets"]} ]'
```

- **API server audit logs** → CloudWatch Logs (enable all log types)
- **Private endpoint only** — API server not accessible from internet
- **Cluster SG** — auto-created, controls API server ↔ node communication

3.2 Worker Node Creation & Joining

WORKER NODE BOOTSTRAP SEQUENCE

1. ASG launches EC2 instance with EKS-optimized AMI
 - └ AMI contains: kubelet, containerd, VPC CNI, SSM
2. Instance userdata runs /etc/eks/bootstrap.sh

```
#!/bin/bash  
/etc/eks/bootstrap.sh my-cluster \  
  --kubelet-extra-args \  
    '--node-labels=env=prod,team=payments \  
    --max-pods=58'
```
3. bootstrap.sh does:
 - a. Discovers API server endpoint via EKS API
 - b. Retrieves CA certificate
 - c. Creates kubeconfig for kubelet
 - d. Configures containerd
 - e. Starts kubelet systemd service
4. kubelet sends CSR to API server
 - └ Node Authorizer validates via aws-auth ConfigMap
5. API server approves → node status = Ready
6. VPC CNI (aws-node DaemonSet) starts on node
 - └ Attaches ENIs, allocates IP from pool
7. kube-proxy DaemonSet starts
 - └ Programs iptables/IPVS rules for Services

```
8. Node is Ready → Scheduler assigns pending pods
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: aws-auth
  namespace: kube-system
data:
  mapRoles: |
    # Node IAM role – allows EC2 instances to join as workers
    - rolearn: arn:aws:iam::123456:role/eks-node-role
      username: system:node:{{EC2PrivateDNSName}}
      groups:
        - system:bootstrappers
        - system:nodes
    # Admin IAM role – human access
    - rolearn: arn:aws:iam::123456:role/eks-admin
      username: admin
      groups:
        - system:masters
    # Developer role – limited access
    - rolearn: arn:aws:iam::123456:role/eks-developer
      username: developer
      groups:
        - dev-group
```

```
┌ Layer 1: VPC Security Groups ────────────┐  
│ Cluster SG: API server ↔ Nodes (auto-managed) │  
│ Node SG:    Nodes ↔ Nodes, Nodes → Aurora (5432) │  
│ Pod SG:     Per-pod SG via SecurityGroupPolicy CRD │  
└───────────────────────────────────────────┘  
  
|  
  
┌ Layer 2: Kubernetes Network Policies (Calico) ────────────┐  
│ Default deny all → explicitly allow required flows │  
└───────────────────────────────────────────┘
```

Layer 3: Service Mesh mTLS (Istio/App Mesh)
Encrypted pod-to-pod communication

Network Policy examples (Calico):

```
# Default deny ALL traffic in namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: payments
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]

---
# Allow payment-service → Aurora (port 5432)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-payment-to-aurora
  namespace: payments
spec:
  podSelector:
    matchLabels:
      app: payment-service
  policyTypes: [Egress]
  egress:
    - to:
        - ipBlock:
            cidr: 10.0.20.0/24    # Aurora subnet CIDR
      ports:
        - protocol: TCP
          port: 5432

---
# Allow payment-service to receive from ingress controller only
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-ingress-to-payment
  namespace: payments
spec:
  podSelector:
    matchLabels:
      app: payment-service
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
```

```

      name: ingress-nginx
    ports:
      - protocol: TCP
        port: 8080

---
# Allow DNS resolution (required for all pods)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
  namespace: payments
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector: {}
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53

```

4.2 Pod Security (Pod Security Standards)

```

# Enforce restricted PSS at namespace level
apiVersion: v1
kind: Namespace
metadata:
  name: payments
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted

```

4.3 IRSA Security for Aurora Access

```

# ServiceAccount with IRSA for RDS IAM Authentication
apiVersion: v1
kind: ServiceAccount
metadata:
  name: payment-service
  namespace: payments

```



```
annotations:
```

```
eks.amazonaws.com/role-arn: arn:aws:iam::123456:role/payment-rds-role
```

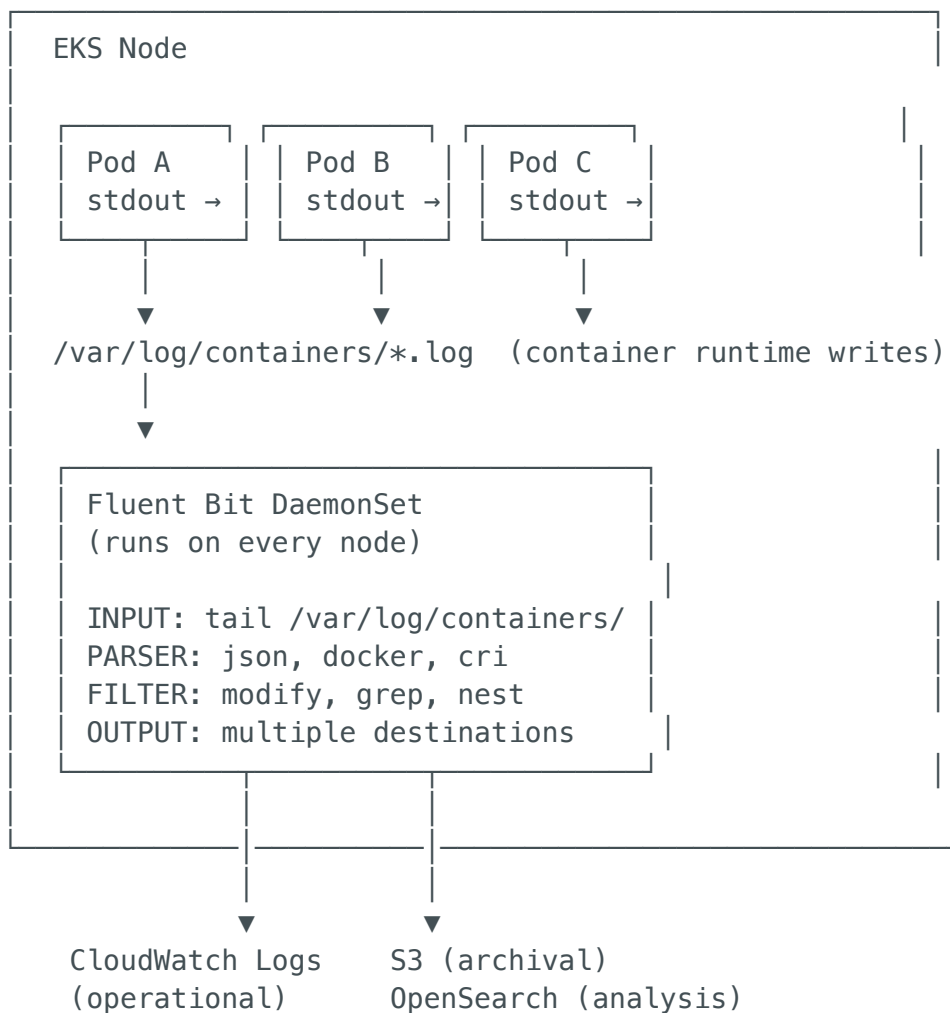
```
# IAM Role trust policy scoped to this SA only
```

```
# IAM Policy: rds-db:connect permission for IAM DB auth
```

```
# No password in Secrets Manager needed – IAM token-based auth
```

5. FLUENT BIT LOG FORWARDING

Architecture



Fluent Bit DaemonSet Configuration

```
apiVersion: v1
kind: ConfigMap
metadata:
```

```
name: fluent-bit-config
namespace: logging
data:
  fluent-bit.conf: |
    [SERVICE]
      Flush          5
      Log_Level      info
      Daemon         off
      Parsers_File   parsers.conf
      HTTP_Server    0n
      HTTP_Listen    0.0.0.0
      HTTP_Port      2020    # Health check endpoint

    [INPUT]
      Name            tail
      Tag             kube.*
      Path            /var/log/containers/*.log
      Parser          cri
      DB              /var/fluent-bit/state/flb_container.db
      Mem_Buf_Limit   50MB
      Skip_Long_Lines 0n
      Refresh_Interval 10

    [FILTER]
      Name            kubernetes
      Match           kube.*
      Kube_URL        https://kubernetes.default.svc:443
      Kube_CA_File    /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
      Kube_Token_File /var/run/secrets/kubernetes.io/serviceaccount/token
      Merge_Log       0n
      K8S-Logging.Parser 0n
      K8S-Logging.Exclude 0n
      Labels         0n
      Annotations    0ff

# Exclude health check logs (noise reduction)
[FILTER]
  Name    grep
  Match   kube.*
  Exclude log /health|/ready|/live/

# Add cluster metadata
[FILTER]
  Name    modify
  Match   kube.*
  Add     cluster my-cluster
  Add     environment production
  Add     region us-east-1

# CloudWatch output (operational logs – 30 day retention)
[OUTPUT]
  Name            cloudwatch_logs
  Match           kube.*
  region         us-east-1
  log_group_name /eks/my-cluster/application
```

```

log_stream_prefix    fluentbit-
auto_create_group    true
log_retention_days   30

# S3 output (archival – lifecycle to Glacier)
[OUTPUT]
  Name                s3
  Match               kube.*
  region              us-east-1
  bucket              my-cluster-logs-archive
  total_file_size     100M
  upload_timeout      10m
  s3_key_format        /logs/%Y/%m/%d/$TAG/%H_%M_%S.gz
  compression         gzip

parsers.conf: |
[PARSER]
  Name                json
  Format              json
  Time_Key            timestamp
  Time_Format         %Y-%m-%dT%H:%M:%S.%LZ

[PARSER]
  Name                cri
  Format              regex
  Regex               ^(?<time>[ ^ ]+) (?<stream>stdout|stderr) (?<logtag>[ ^
]*) (?<log>.*)$
  Time_Key            time
  Time_Format         %Y-%m-%dT%H:%M:%S.%L%z

```

Fluent Bit DaemonSet Deployment

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
  namespace: logging
spec:
  selector:
    matchLabels:
      app: fluent-bit
  template:
    metadata:
      labels:
        app: fluent-bit
    spec:
      serviceAccountName: fluent-bit # IRSA for CloudWatch + S3 access
      tolerations:
        - operator: Exists # Run on ALL nodes including tainted
      containers:
        - name: fluent-bit
          image: public.ecr.aws/aws-observability/aws-for-fluent-bit:2.32.0

```

```

resources:
  limits:
    memory: 200Mi
    cpu: 200m
  requests:
    memory: 100Mi
    cpu: 100m
  volumeMounts:
    - name: varlog
      mountPath: /var/log
      readOnly: true
    - name: config
      mountPath: /fluent-bit/etc/
    - name: state
      mountPath: /var/fluent-bit/state
  ports:
    - containerPort: 2020 # Metrics/health
  livenessProbe:
    httpGet:
      path: /api/v1/health
      port: 2020
    initialDelaySeconds: 10
  readinessProbe:
    httpGet:
      path: /api/v1/health
      port: 2020
  volumes:
    - name: varlog
      hostPath:
        path: /var/log
    - name: config
      configMap:
        name: fluent-bit-config
    - name: state
      hostPath:
        path: /var/fluent-bit/state

```

Fluent Bit IRSA (Private Cluster — No Internet)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents",
        "logs:DescribeLogStreams"
      ],
      "Resource": "arn:aws:logs:us-east-1:123456:log-group:/eks/my-cluster/*"
    }
  ]
}

```

```

    },
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject"],
      "Resource": "arn:aws:s3:::my-cluster-logs-archive/*"
    }
  ]
}

```

Required VPC endpoints for Fluent Bit in private cluster:

- [com.amazonaws.us-east-1.logs](#) — CloudWatch Logs
- [com.amazonaws.us-east-1.s3](#) — S3 (gateway endpoint)

Fluent Bit During EKS Upgrade

Concern	Impact	Mitigation
DaemonSet pod restarts on new nodes	Brief log gap (seconds)	DB file tracks position — resumes from last read
Old node drained before logs flushed	Last few log lines lost	Set terminationGracePeriodSeconds: 60 on Fluent Bit
Image pull in private cluster	Fluent Bit image not available	Pre-pull image to ECR: public.ecr.aws/aws-observability/aws-for-fluent-bit → your ECR
VPC endpoint missing	Logs stop forwarding silently	Verify logs VPC endpoint exists before upgrade

Part 3 of EKS Upgrade Series — Networking, Node Lifecycle, Security & Fluent Bit
Context: Private EKS Cluster + Aurora PostgreSQL + Calico Network Policies